

Mining Data Streams

Partially based on slides from:

<http://infolab.stanford.edu/~ullman/mining/2009/index.html>

Anand Rajaraman & Jeff Ullman

Contents

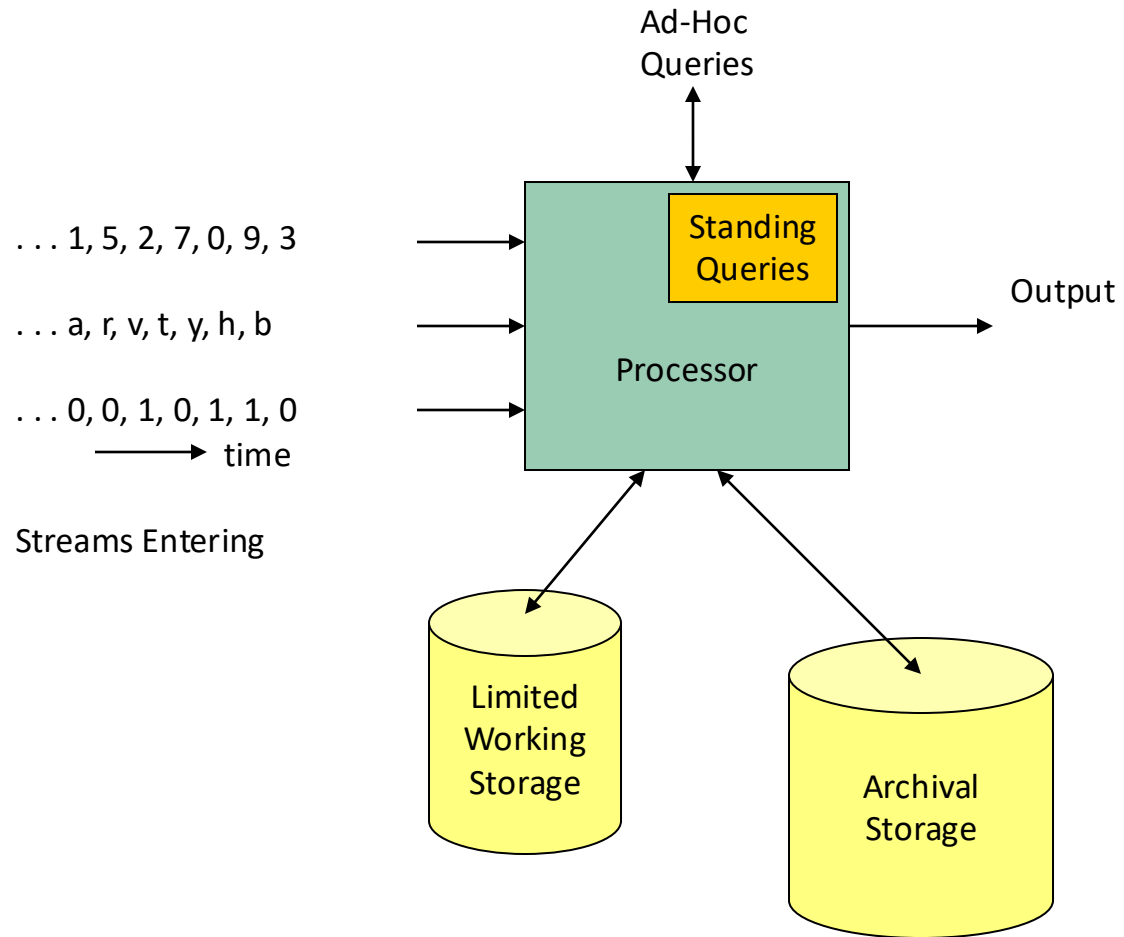
- Data Stream Model
- Random sampling of data streams (Reservoir Sampling)
- Filtering data streams (Bloom Filter)
- Counting distinct elements in a stream

Conventional Data Mining

- All data stored on slow hard disks/archives
- Slow access, slow processing
- Off-line analysis of data
- Instant action on results impossible
- However, trained MODELS (rules, decision trees, neural networks, etc.) can be deployed to work in real time

The Stream Model

- Data enters the system at a rapid rate
- The system cannot store the entire stream accessibly
- How do you make critical calculations about the stream using a limited amount of fast (RAM) memory?
- We might be interested in queries over:
 - the last N records (a sliding window model)
 - everything seen so far (or the last N days)



Applications (1)

- Mining Twitter/X's tweets
 - Detecting breaking news, disasters, scandals, ...
- Mining query streams
 - Google wants to know what queries are more frequent today than yesterday
 - Google Flu Trends
- Mining click streams
 - Yahoo wants to know which of its pages are getting an unusual number of hits in the past hour

Applications (2)

- Sensors of all kinds need monitoring, especially when there are many sensors of the same type, feeding into a central controller
 - Detecting/predicting traffic jams
 - Tsunami alerts
- Telephone call records are summarized into customer bills
- IP packets can be monitored at a switch
 - Gather information for optimal routing
 - Detect denial-of-service attacks
 - Filter spam or phishing attacks

Sampling a Data Stream

Sampling a Data Stream

- Suppose a bank would like to have (at any moment) a ‘representative’ sample of 2% of all their data, so it would fit into RAM and could be queried interactively (quick response)
- Example query:
what is the ratio of clients with exactly 1 transaction to clients who made exactly 2 transactions during the last weekend?

Naive Approach

- Scan the stream of all transactions; the n -th transaction is put into the sample iff

$$r = (n \bmod 50) = 0$$

(0, 50, 100, 150, ...)

- Use a random selection with $p = 0.02$
- Will not work:
 - Customers with 2, 3, ... transactions would look like having only a single transaction

Better Approach

- Make, off-line, a list of all clients, take a 2% sample of them, put them into a hash table (or a binary search tree) and store only transactions from these selected clients
- Problems:
 - Cost time and memory (to keep the hash table)
 - What about new clients? List of clients is fixed once you build an off-line list

An Even Better Approach

- Instead of a hash table, use a hash *function* to map each client to an integer 0, ..., 49. Only transactions of clients with number 0 enter the sample
- Advantage:
 - No need to create and keep any hash table
 - New clients are also included in the sampling process
- Problem: what if the sample size exceeds available RAM?
 - Perhaps 2% is not a static number

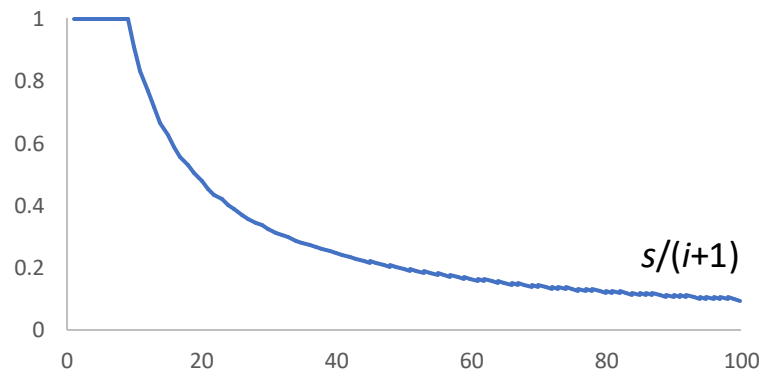
Reservoir Sampling

- Suppose your RAM can store **s records**
- How could you sample an infinite stream of records?
 - at any moment, your RAM keeps a **random, uniformly distributed**, sample of s records?
 - every record from the stream has the same probability s/n of being kept in RAM

Jeffrey Vitter, Random sampling with a reservoir. *ACM Transactions on Mathematical Software* 11 (1), 1985
derived from Knuth, *The Art of Computer Programming*, Vol. 2, §3.4.2

Reservoir Sampling

- Initialization:
Store the first s records of the stream in your RAM. At this moment, $n = s$ and the probability of an element entering RAM is $s/n = 1$
- Inductive Step:
 - When the $(i+1)^{\text{th}}$ element arrives, decide with probability $s/(i+1)$ to keep the record in RAM. Otherwise, ignore it
 - If you choose to keep it, throw one of the previously stored records out, selected with equal probability, and use the free space for the new record
- Prove by induction that at any moment all records enter RAM with probability s/i



Algorithm Runthrough

 keep

1 

2 

...

10 

s

11 

 random

$p=s/(i+1)=10/11=0.909$

12 

 random

$p=s/(i+1)=10/12=0.833$

13 

$p=s/(i+1)=10/13=0.769$

...

20 

Proof

- Apply the algorithm to the $(i + 1)^{\text{th}}$ input
- For any existing x_a , the probability that it is included in the reservoir before is $\frac{s}{i}$
- Input x_{i+1} is included with probability $\frac{s}{i+1}$ by definition of the algorithm
- The probability that x_a is still included in the reservoir after x_{i+1} is processed is:

$$\begin{aligned} & \frac{s}{i} \cdot \left(1 - \frac{1}{i+1}\right) = \\ & \frac{s}{i} - \frac{s}{i(i+1)} = \\ & \frac{s(i+1)}{i(i+1)} - \frac{s}{i(i+1)} = \\ & \frac{s(i+1-1)}{i(i+1)} = \frac{s}{i+1} \end{aligned}$$

probability x_a was in the reservoir

probability x_a has survived:
 $1 - \frac{s}{i+1} \cdot \frac{1}{s} = 1 - \frac{1}{i+1}$

probability of replacement

probability it concerns x_a

Break

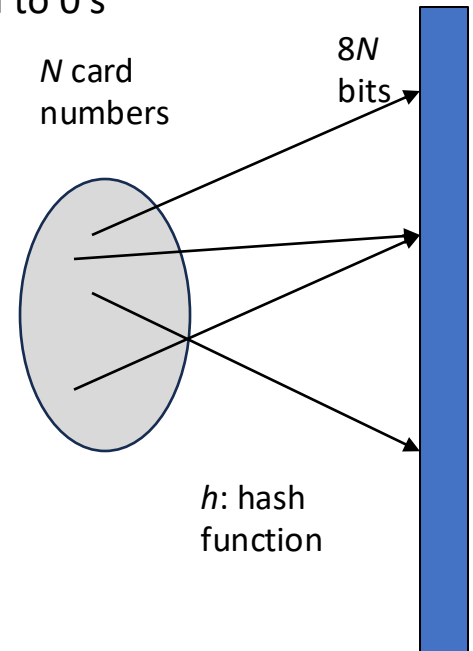
Bloom Filters

Fast Filtering of 'Bad' Records

- Sometimes we know which records are 'legal' and want to filter out all (or almost all) 'illegal' records from a stream
- Examples:
 - A bank wants to ignore card numbers of clients that are not in their database
 - A mailing system wants to block all incoming e-mails from 'non-registered' e-mail addresses (spam?)
 - Amazon.com wants to allow access only to their registered clients
- Challenge: find a data structure that
 - is guaranteed to report 'legal', when testing a legal record
 - will report 'illegal' most of the time, when testing an illegal record
 - false positive
 - waste some time further checking an illegal record
 - is not too big
 - is fast

Bloom Filter

- Scenario: $N = 1$ billion 'legal' card numbers, each 28 chars long; how could we quickly filter out almost all 'illegal' numbers?
- A lookup table: **56 GB RAM**
- **Bloom Filter (one hash function):**
 - Consider a vector of **8 billion bits** (just **1 GB of RAM!**) initialized to 0's and a hash function $h(x)$ with values in $1, \dots, 8 \cdot 10^9$
 - For each 'legal' card number x , **set bit $h(x)$ to 1**
 - An incoming y is considered to be legal if the $h(y)$ bit is 1
- What is the percentage of **false positives**?
(illegal cards that would pass through the filter)



Bloom Filter: Rate of False Positives

- How many bits are set to 1?
 - $n = 10^9$ legal card numbers
 - $N = 8 \cdot 10^9$ buckets
- What is the chance that a randomly selected bit is **still 0**?

$$\left(1 - \frac{1}{N}\right)^n$$

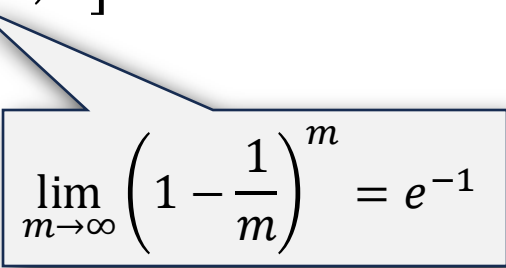
Bloom Filter: Rate of False Positives

- How many bits are set to 1?
 - $n = 10^9$ legal card numbers
 - $N = 8 \cdot 10^9$ buckets

- What is the chance that a randomly selected bit is **still 0**?

$$\left(1 - \frac{1}{N}\right)^n = \left[\left(1 - \frac{1}{N}\right)^N\right]^{n/N} \approx e^{-1/8} = 0.8825$$

- Thus the chance of randomly hitting positives is about **11.75%**


$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m = e^{-1}$$

so the rate of false

Bloom Filter **with k Hash Functions**

- How many bits are set to 1?
 - $n = 10^9$ legal card numbers
 - $N = 8 \cdot 10^9$ buckets
 - k hash functions, all k buckets are set to 1
 - for querying, all k buckets need to be 1 for a 'legal' record

- What is the chance that a randomly selected bit is **still 0**?

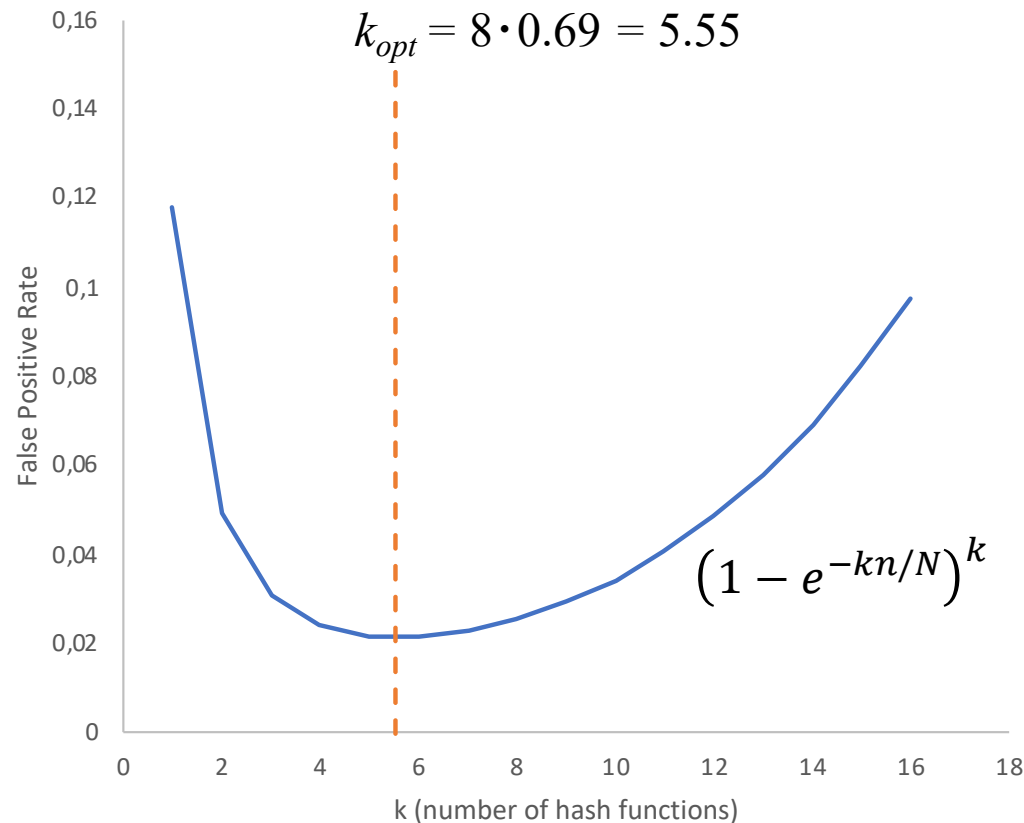
$$\left(1 - \frac{1}{N}\right)^{kn} = \left[\left(1 - \frac{1}{N}\right)^N\right]^{kn/N} \approx e^{-kn/N}$$

- Thus the chance of hitting 1 by k independent hash functions is:

$$\left(1 - e^{-kn/N}\right)^k$$

- $k = 2$, $n/N = 1/8$: false positive rate = $\left(1 - e^{-1/4}\right)^2 = 0.0493 \approx 5\%$

Optimal Number of Hash Functions



What number of hash functions k is optimal? $k_{opt} = (N/n) \cdot \ln 2$

Final Remarks on Bloom Filters

- Inserting new elements in a filter is easy
- Removal of elements from a filter is almost impossible
- Cascading: using two or more filters one after another reduces errors 'exponentially fast' (e.g., $5\% \cdot 5\% = 0.25\%$)

Applications of Bloom Filters

- **Google BigTable** and **Apache Cassandra** use Bloom filters to reduce the disk lookups for non-existent rows or columns. Avoiding costly disk lookups considerably increases the performance of a database query operation
- The **Google Chrome** web browser used to use a Bloom filter to identify malicious URLs. Any URL was first checked against a local Bloom filter, and only if the Bloom filter returned a positive result was a full check of the URL performed (and the user warned, if that too returned a positive result)
- The **Squid Web Proxy Cache** uses Bloom filters for cache digests
- **Bitcoin** uses Bloom filters to speed up wallet synchronization
- The **Venti** archival storage system uses Bloom filters to detect previously stored data